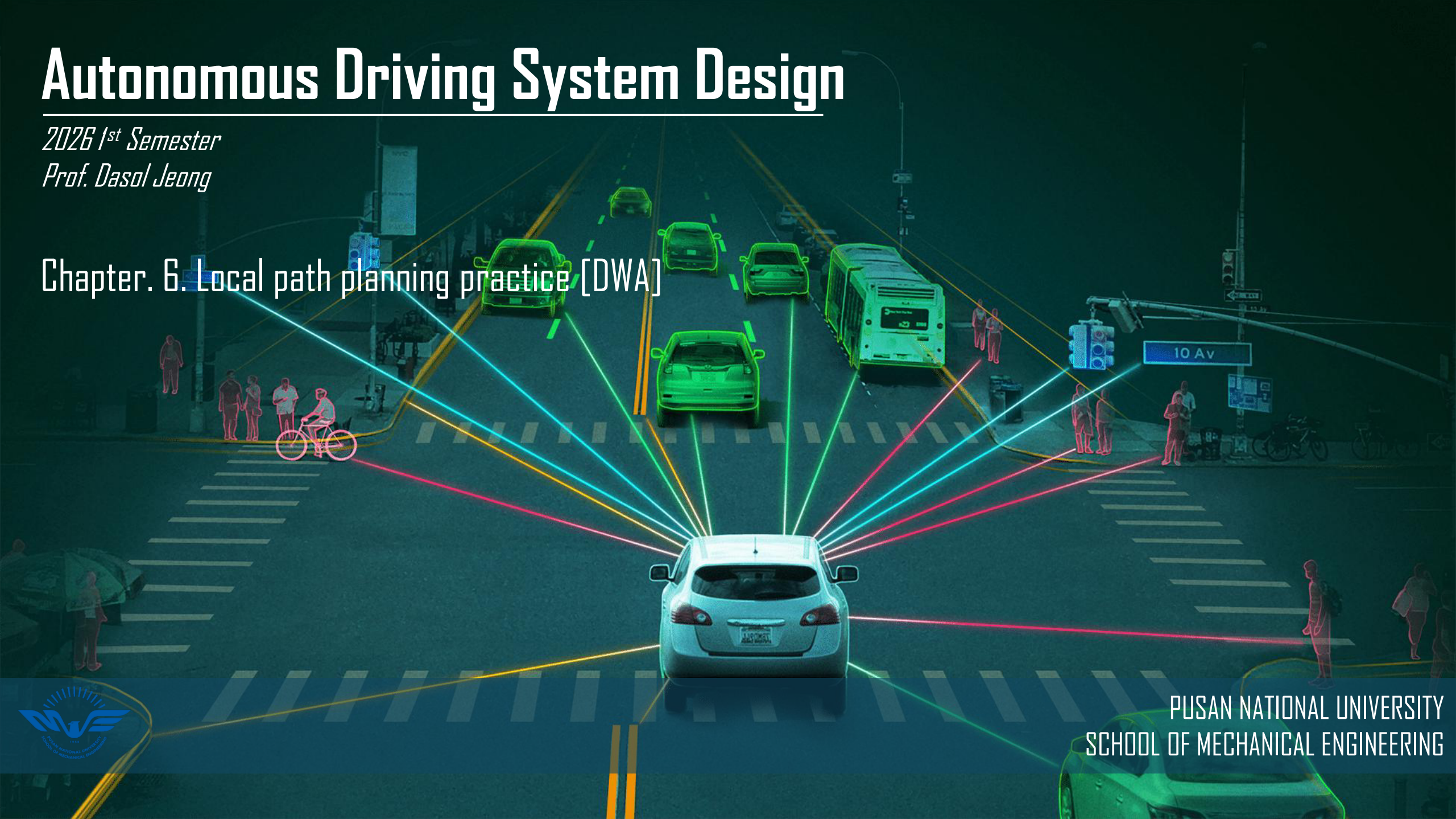
Autonomous Driving System Design

2026 1st Semester

Prof. Dasol Jeong



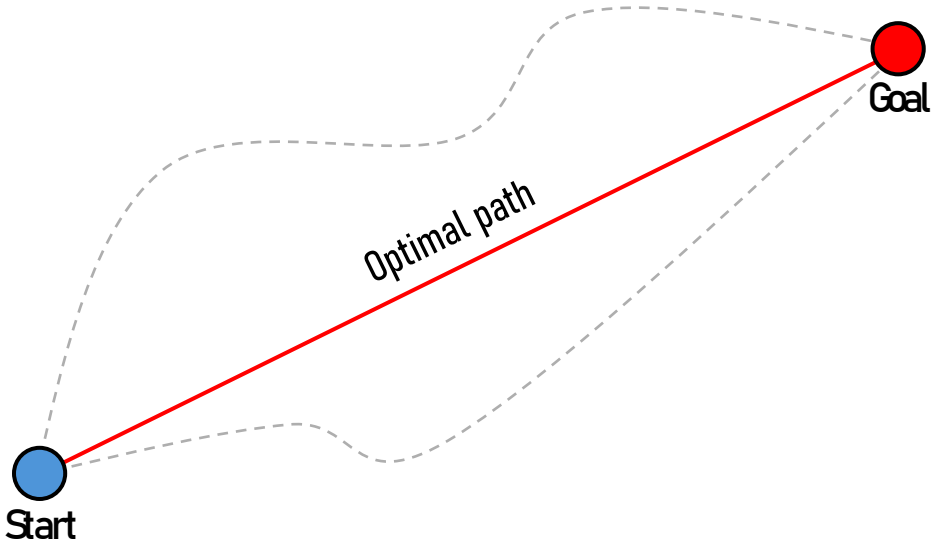
Chapter. 6. Local path planning practice (DWA)

PUSAN NATIONAL UNIVERSITY
SCHOOL OF MECHANICAL ENGINEERING

Path planning

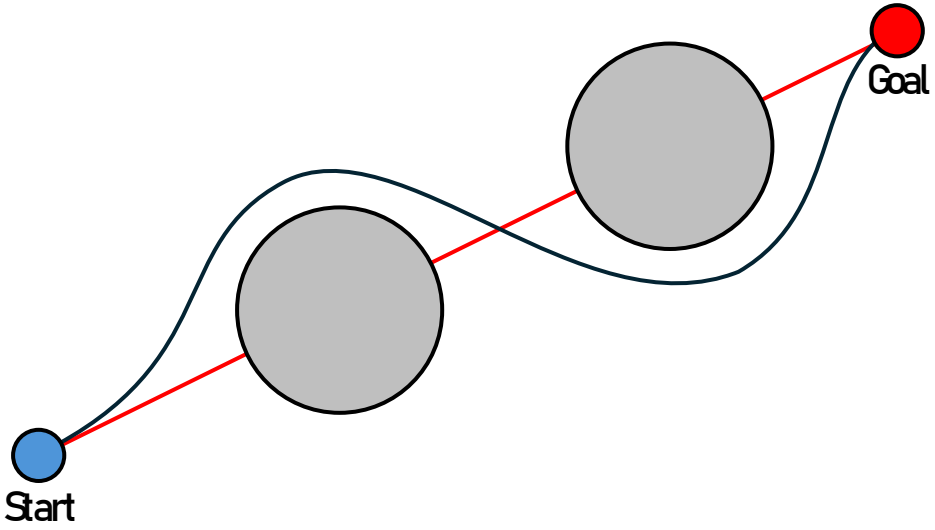
Global path planning

- Plans the overall route from start to goal
- Uses prior knowledge of the environment (global maps)
- Examples: Dijkstra , A*, RRT, RRT*



Local path planning

- Handles real-time navigation and obstacle avoidance
- Uses sensor data to detect immediate surroundings
- Must respond quickly to dynamic environment changes
- Examples: VFH, APF, DWA

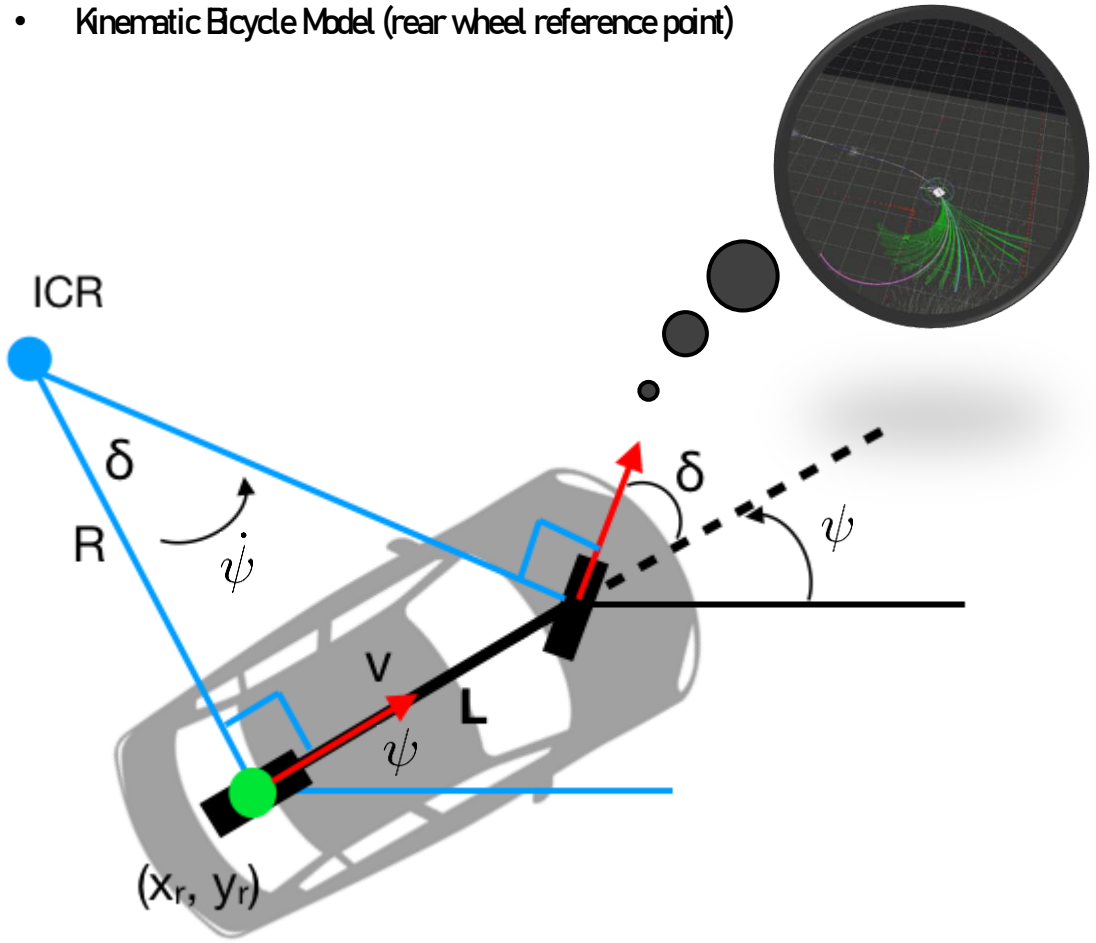


Local path planning – DWA pseudocode

❖ File directory: pnu-class-distribution/class_materials/dwa_package/controller/algorithm/dwa_core.py

TODO1

- Kinematic Bicycle Model (rear wheel reference point)



- Equation

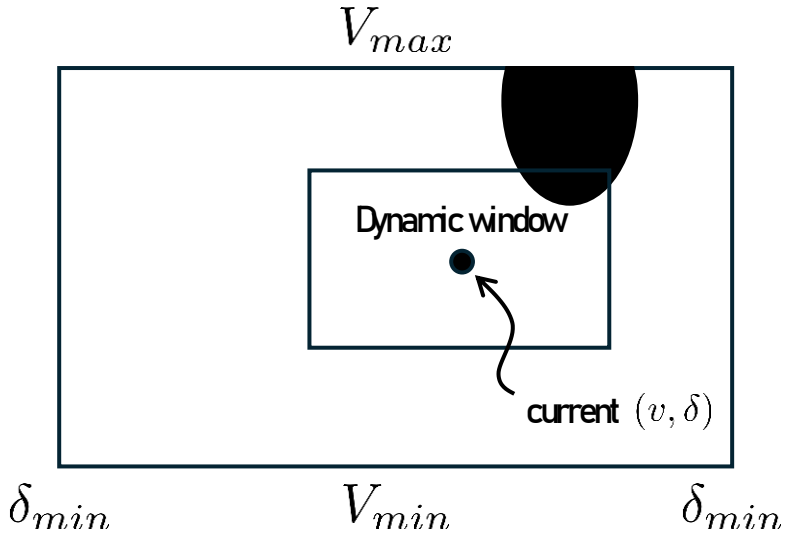
assumptions:

1. No tire slip
2. Zero steer angle at rear wheel

$$V_X = V\cos(\psi)$$
$$V_Y = V\sin(\psi)$$
$$\dot{\psi} = \frac{V}{R} = \frac{V\tan\delta}{L}$$

Local path planning – DWA pseudocode

TODO 2



Input: $X_{current}, Goal, Obstacles$

Output: v_{opt}, δ_{opt}

1 : $v_{min} \leftarrow \max(V_{min}, v_{current} - a_{max} \cdot \Delta t)$

2 : $v_{max} \leftarrow \min(V_{max}, v_{current} + a_{max} \cdot \Delta t)$

3 : $\delta_{min} \leftarrow \max(-\delta_{max}, \delta_{current} - \dot{\delta}_{max} \cdot \Delta t)$

4 : $\delta_{max} \leftarrow \min(\delta_{max}, \delta_{current} + \dot{\delta}_{max} \cdot \Delta t)$

5 : $Dw \leftarrow [v_{min}, v_{max}, \delta_{min}, \delta_{max}]$

6 : $G_{best} \leftarrow \infty,\ v_{opt} \leftarrow 0,\ \delta_{opt} \leftarrow 0$

7 : **for** $v = v_{min}$ to v_{max} step Δv **do**

8 : **for** $\delta = \delta_{min}$ to δ_{max} step $\Delta\delta$ **do**

9 : $Traj \leftarrow \text{PredictTrajectory}(X_{current}, v, \delta, T_{pred})$

10: **if** $\text{CollisionCheck}(Traj, Obstacles)$ **then continue**

11: $G_{heading} \leftarrow \text{HeadingCost}(Traj, Goal)$

12: $G_{dist} \leftarrow \text{ObstacleCost}(Traj, Obstacles)$

13: $G_{vel} \leftarrow \text{VelocityCost}(v)$

14: $G \leftarrow \alpha \cdot G_{heading} + \beta \cdot G_{dist} + \gamma \cdot G_{vel}$

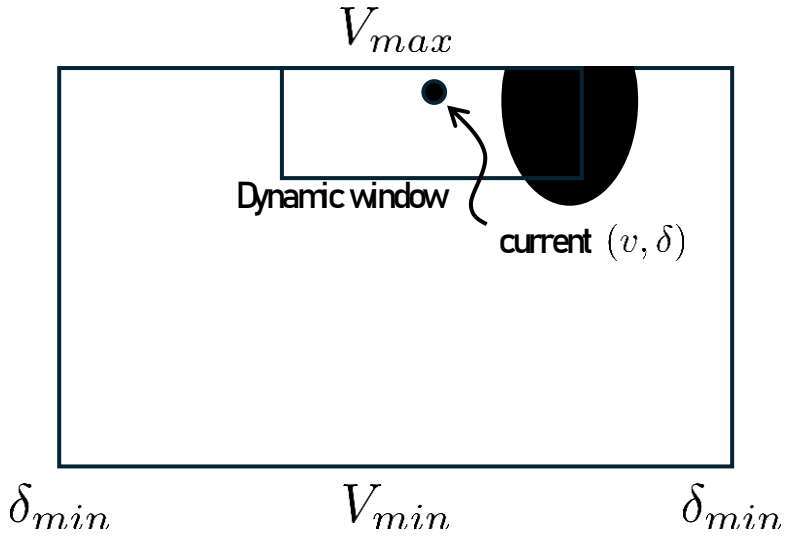
15: **if** $G < G_{best}$ **then**

16: $G_{best} \leftarrow G,\ v_{opt} \leftarrow v,\ \delta_{opt} \leftarrow \delta$

17: **return** v_{opt}, δ_{opt}

Local path planning – DWA pseudocode

TODO 2



Input: $X_{current}, Goal, Obstacles$

Output: v_{opt}, δ_{opt}

1 : $v_{min} \leftarrow \max(V_{min}, v_{current} - a_{max} \cdot \Delta t)$

2 : $v_{max} \leftarrow \min(V_{max}, v_{current} + a_{max} \cdot \Delta t)$

3 : $\delta_{min} \leftarrow \max(-\delta_{max}, \delta_{current} - \dot{\delta}_{max} \cdot \Delta t)$

4 : $\delta_{max} \leftarrow \min(\delta_{max}, \delta_{current} + \dot{\delta}_{max} \cdot \Delta t)$

5 : $Dw \leftarrow [v_{min}, v_{max}, \delta_{min}, \delta_{max}]$

6 : $G_{best} \leftarrow \infty,\ v_{opt} \leftarrow 0,\ \delta_{opt} \leftarrow 0$

7 : **for** $v = v_{min}$ to v_{max} step Δv **do**

8 : **for** $\delta = \delta_{min}$ to δ_{max} step $\Delta\delta$ **do**

9 : $Traj \leftarrow \text{PredictTrajectory}(X_{current}, v, \delta, T_{pred})$

10: **if** $\text{CollisionCheck}(Traj, Obstacles)$ **then continue**

11: $G_{heading} \leftarrow \text{HeadingCost}(Traj, Goal)$

12: $G_{dist} \leftarrow \text{ObstacleCost}(Traj, Obstacles)$

13: $G_{vel} \leftarrow \text{VelocityCost}(v)$

14: $G \leftarrow \alpha \cdot G_{heading} + \beta \cdot G_{dist} + \gamma \cdot G_{vel}$

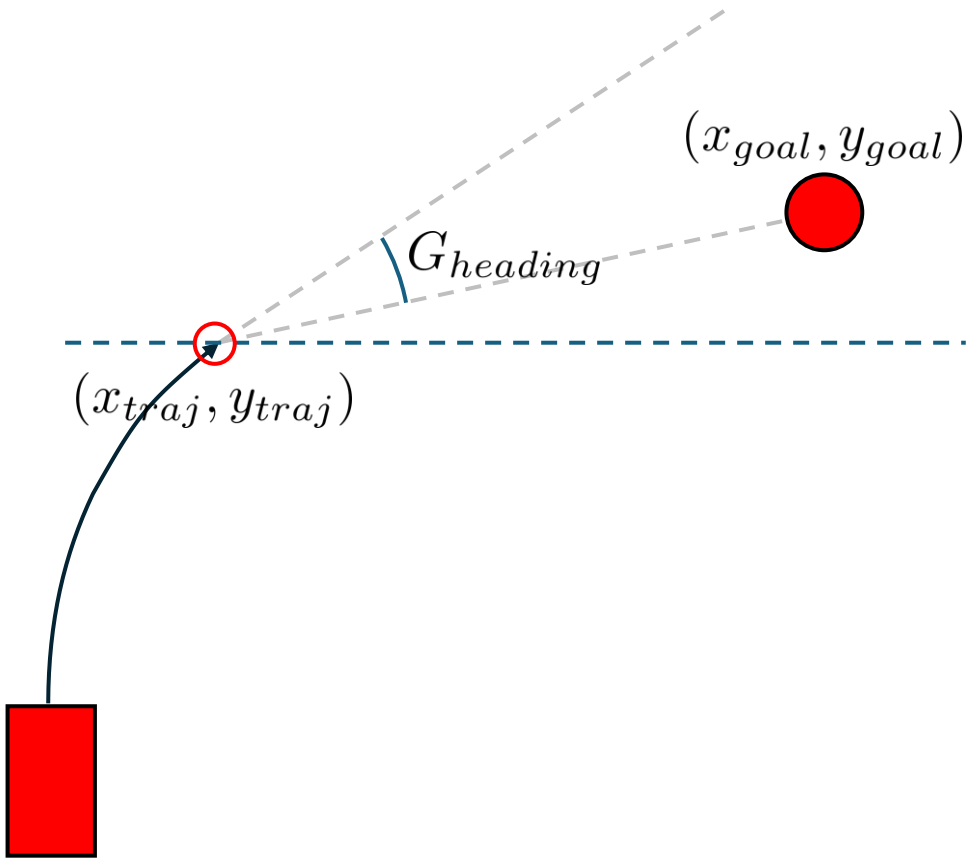
15: **if** $G < G_{best}$ **then**

16: $G_{best} \leftarrow G,\ v_{opt} \leftarrow v,\ \delta_{opt} \leftarrow \delta$

17: **return** v_{opt}, δ_{opt}

Local path planning – DWA pseudocode

TODO 3



Cost: $G(v,\delta) = \rho(\alpha \cdot G_{heading}(v,\delta) + \beta \cdot G_{dist}(v,\delta) + \gamma \cdot G_{vel}(v,\delta))$

Input: $X_{current}, Goal, Obstacles$

Output: v_{opt}, δ_{opt}

$v_{min} \leftarrow \max(V_{min}, v_{current} - a_{max} \cdot \Delta t)$

$v_{max} \leftarrow \min(V_{max}, v_{current} + a_{max} \cdot \Delta t)$

$\delta_{min} \leftarrow \max(-\delta_{max}, \delta_{current} - \dot{\delta}_{max} \cdot \Delta t)$

$\delta_{max} \leftarrow \min(\delta_{max}, \delta_{current} + \dot{\delta}_{max} \cdot \Delta t)$

$Dw \leftarrow [v_{min}, v_{max}, \delta_{min}, \delta_{max}]$

$G_{best} \leftarrow \infty,\ v_{opt} \leftarrow 0,\ \delta_{opt} \leftarrow 0$

for $v = v_{min}$ to v_{max} step Δv **do**

  for $\delta = \delta_{min}$ to δ_{max} step $\Delta\delta$ **do**

    $Traj \leftarrow \text{PredictTrajectory}(X_{current}, v, \delta, T_{pred})$

    if $\text{CollisionCheck}(Traj, Obstacles)$ **then continue**

    $G_{heading} \leftarrow \text{HeadingCost}(Traj, Goal)$

    $G_{dist} \leftarrow \text{ObstacleCost}(Traj, Obstacles)$

    $G_{vel} \leftarrow \text{VelocityCost}(v)$

    $G \leftarrow \alpha \cdot G_{heading} + \beta \cdot G_{dist} + \gamma \cdot G_{vel}$

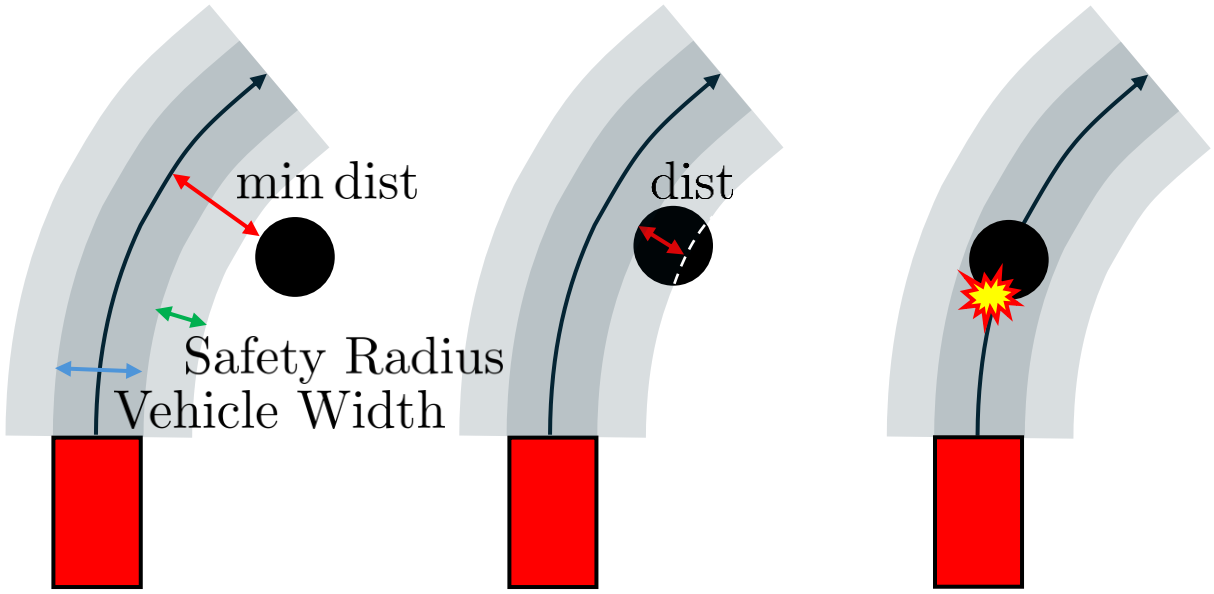
    if $G < G_{best}$ **then**

      $G_{best} \leftarrow G,\ v_{opt} \leftarrow v,\ \delta_{opt} \leftarrow \delta$

return v_{opt}, δ_{opt}

Local path planning – DWA pseudocode

TODO 4



$G_{dist} = 0$ $G_{dist} = \exp(\text{dist})$ $G_{dist} = \text{inf}$

Cost: $G(v,\delta) = \rho(\alpha \cdot G_{heading}(v,\delta) + \beta \cdot \boxed{G_{dist}(v,\delta)} + \gamma \cdot G_{vel}(v,\delta))$

Input: $X_{current}, Goal, Obstacles$

Output: v_{opt}, δ_{opt}

$v_{min} \leftarrow \max(V_{min}, v_{current} - a_{max} \cdot \Delta t)$

$v_{max} \leftarrow \min(V_{max}, v_{current} + a_{max} \cdot \Delta t)$

$\delta_{min} \leftarrow \max(-\delta_{max}, \delta_{current} - \dot{\delta}_{max} \cdot \Delta t)$

$\delta_{max} \leftarrow \min(\delta_{max}, \delta_{current} + \dot{\delta}_{max} \cdot \Delta t)$

$Dw \leftarrow [v_{min}, v_{max}, \delta_{min}, \delta_{max}]$

$G_{best} \leftarrow \infty,\ v_{opt} \leftarrow 0,\ \delta_{opt} \leftarrow 0$

for $v = v_{min}$ to v_{max} step Δv **do**

  for $\delta = \delta_{min}$ to δ_{max} step $\Delta\delta$ **do**

    $Traj \leftarrow \text{PredictTrajectory}(X_{current}, v, \delta, T_{pred})$

    if $\text{CollisionCheck}(Traj, Obstacles)$ **then continue**

    $G_{heading} \leftarrow \text{HeadingCost}(Traj, Goal)$

    $\boxed{G_{dist} \leftarrow \text{ObstacleCost}(Traj, Obstacles)}$

    $G_{vel} \leftarrow \text{VelocityCost}(v)$

    $G \leftarrow \alpha \cdot G_{heading} + \beta \cdot G_{dist} + \gamma \cdot G_{vel}$

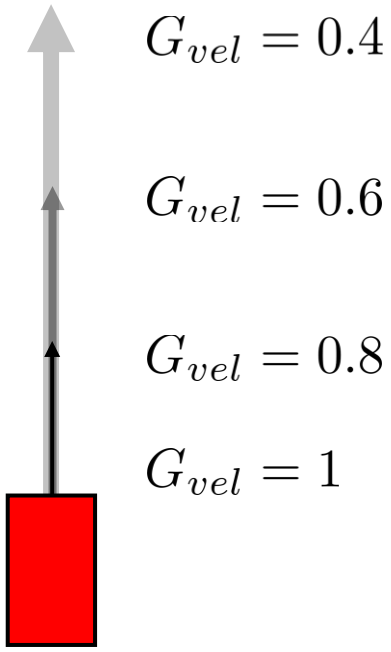
    if $G < G_{best}$ **then**

      $G_{best} \leftarrow G,\ v_{opt} \leftarrow v,\ \delta_{opt} \leftarrow \delta$

return v_{opt}, δ_{opt}

Local path planning – DWA pseudocode

TODO 5



Cost: $G(v, \delta) = \rho(\alpha \cdot G_{heading}(v, \delta) + \beta \cdot G_{dist}(v, \delta) + \gamma \cdot G_{vel}(v, \delta))$

Input: $X_{current}, Goal, Obstacles$

Output: v_{opt}, δ_{opt}

$v_{min} \leftarrow \max(V_{min}, v_{current} - a_{max} \cdot \Delta t)$

$v_{max} \leftarrow \min(V_{max}, v_{current} + a_{max} \cdot \Delta t)$

$\delta_{min} \leftarrow \max(-\delta_{max}, \delta_{current} - \dot{\delta}_{max} \cdot \Delta t)$

$\delta_{max} \leftarrow \min(\delta_{max}, \delta_{current} + \dot{\delta}_{max} \cdot \Delta t)$

$Dw \leftarrow [v_{min}, v_{max}, \delta_{min}, \delta_{max}]$

$G_{best} \leftarrow \infty,\ v_{opt} \leftarrow 0,\ \delta_{opt} \leftarrow 0$

for $v = v_{min}$ to v_{max} step Δv **do**

  for $\delta = \delta_{min}$ to δ_{max} step $\Delta\delta$ **do**

    $Traj \leftarrow \text{PredictTrajectory}(X_{current}, v, \delta, T_{pred})$

    if $\text{CollisionCheck}(Traj, Obstacles)$ **then continue**

    $G_{heading} \leftarrow \text{HeadingCost}(Traj, Goal)$

    $G_{dist} \leftarrow \text{ObstacleCost}(Traj, Obstacles)$

    $G_{vel} \leftarrow \text{VelocityCost}(v)$

    $G \leftarrow \alpha \cdot G_{heading} + \beta \cdot G_{dist} + \gamma \cdot G_{vel}$

    if $G < G_{best}$ **then**

      $G_{best} \leftarrow G,\ v_{opt} \leftarrow v,\ \delta_{opt} \leftarrow \delta$

return v_{opt}, δ_{opt}

Running CARLA with ROS Bridge

1. Start CARLA server

```
user@ubuntu:~/
user@ubuntu:~$ cd pnu-class-distribution
user@ubuntu:~/pnu-class-distribution/$ cd SDL-CARLA-ENV
user@ubuntu:~/pnu-class-distribution/SDL-CARLA-ENV$ ./CarlaUE4.sh
4.26.2-0+++UE4+Release-4.26 522 0
Disabling core dumps.
```

2. Build the package

```
user@ubuntu:~/
user@ubuntu:~$ cd pnu-class-distribution/SDL-CARLA-ROS-BRIDGE
user@ubuntu:~/pnu-class-distribution/SDL-CARLA-ROS-BRIDGE$ colcon build
user@ubuntu:~/pnu-class-distribution/SDL-CARLA-ROS-BRIDGE$ source install/setup.bash
```

3. Launch the CARLA ROS Bridge Manual Control

```
user@ubuntu:~/
user@ubuntu:~$ ros2 launch carla_ros_bridge carla_ros_bridge_dwa_custom_map.launch.py
```

Running CARLA with ROS Bridge

4. run rviz2

```
user@ubuntu:~/
user@ubuntu:~$ rviz2
```

5. run global path planning algorithm

```
user@ubuntu:~/
user@ubuntu:~$ cd pnu-class-distribution
user@ubuntu:~/pnu-class-distribution/$ cd class_materials/dwa_package/path_planner
user@ubuntu:~/pnu-class-distribution/class_materials/dwa_package/path_planner$ python3 main_astar.pyc
or
user@ubuntu:~/pnu-class-distribution/class_materials/dwa_package/path_planner$ python3 main_rrt.pyc
or
user@ubuntu:~/pnu-class-distribution/class_materials/dwa_package/path_planner$ python3 main_rrtstar.pyc
```

6. run dwa algorithm

```
user@ubuntu:~/
user@ubuntu:~$ cd pnu-class-distribution
user@ubuntu:~/pnu-class-distribution/$ cd class_materials/dwa_package/controller
user@ubuntu:~/pnu-class-distribution/class_materials/dwa_package/controller$ python3 dwa_controller.pyc
```